

GDA Dashboard - Technical Specification for Production

System Architecture

Frontend Architecture

```
src/
├── components/
│   ├── MilestoneCard.jsx
│   ├── SystemCard.jsx
│   ├── TaskActivityFeed.jsx
│   ├── DependencyVisualization.jsx
│   └── DependencySummary.jsx
├── hooks/
│   ├── useTasks.js
│   ├── useMilestones.js
│   ├── useAuth.js
│   └── useWebSocket.js
├── services/
│   ├── api.js
│   ├── websocket.js
│   └── auth.js
├── utils/
│   ├── dependencyResolver.js
│   └── autoAssigner.js
```

```
|   └─ timeCalculator.js
└─ App.jsx
```

Backend Microservices

1. Auth Service (Node.js + Okta)

Responsibilities:

- Okta SSO integration
- JWT token generation/validation
- Role-based access control (RBAC)

Endpoints:

```
POST /auth/login
POST /auth/logout
POST /auth/refresh
GET  /auth/user
GET  /auth/permissions
```

2. Task Service (Python FastAPI)

Responsibilities:

- CRUD operations on tasks
- Status management
- Assignment logic execution

Endpoints:

```

GET      /api/v1/tasks
POST     /api/v1/tasks
GET      /api/v1/tasks/:id
PUT      /api/v1/tasks/:id
PATCH   /api/v1/tasks/:id/status
POST     /api/v1/tasks/:id/assign
GET      /api/v1/tasks/by-user/:userId
GET      /api/v1/tasks/by-system/:systemId

```

Database Schema (PostgreSQL):

```

CREATE TABLE tasks (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    task_code VARCHAR(50) NOT NULL,
    system_id UUID REFERENCES systems(id),
    milestone_id UUID REFERENCES milestones(id),
    description TEXT NOT NULL,
    timing VARCHAR(20),
    location VARCHAR(100),
    status VARCHAR(20) CHECK (status IN ('Open', 'In Progress',
'Complete', 'Blocked', 'Fail')),
    assigned_role_id UUID REFERENCES roles(id),
    assigned_user_id UUID REFERENCES users(id),
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW(),
    completed_at TIMESTAMP
);

CREATE TABLE task_dependencies (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    task_id UUID REFERENCES tasks(id),
    depends_on_task_id UUID REFERENCES tasks(id),
    created_at TIMESTAMP DEFAULT NOW(),
    UNIQUE(task_id, depends_on_task_id)
);

CREATE INDEX idx_tasks_status ON tasks(status);
CREATE INDEX idx_tasks_assigned_user ON tasks(assigned_user_id);
CREATE INDEX idx_tasks_milestone ON tasks(milestone_id);

```

```
CREATE INDEX idx_task_deps ON task_dependencies(task_id);
```

3. Dependency Service (Python + Neo4j Graph DB)

Responsibilities:

- Dependency graph management
- Cascade resolution algorithm
- Critical path analysis

Endpoints:

```
GET    /api/v1/dependencies/task/:taskId
POST   /api/v1/dependencies
DELETE /api/v1/dependencies/:id
GET    /api/v1/dependencies/critical-path
POST   /api/v1/dependencies/resolve
GET    /api/v1/dependencies/blocked-by/:taskId
```

Neo4j Graph Schema:

```
// Node: Task
CREATE (t:Task {
  id: 'uuid',
  code: 'IVRS_1.0',
  system: 'IVRS',
  status: 'Blocked'
})

// Relationship: DEPENDS_ON
CREATE (t1:Task)-[:DEPENDS_ON]->(t2:Task)

// Query: Find all blocked tasks
MATCH (t:Task {status: 'Blocked'})-[:DEPENDS_ON]->(dep:Task)
WHERE dep.status <> 'Complete'
```

```
RETURN t, dep
```

```
// Query: Find critical path (longest dependency chain)
MATCH path = (t:Task)-[:DEPENDS_ON*]->(root:Task)
WHERE NOT (root)-[:DEPENDS_ON]->()
RETURN path ORDER BY length(path) DESC LIMIT 1
```

4. Timeline Service (Go)

Responsibilities:

- Milestone scheduling
- Kickoff time calculations
- Task activation triggers

Endpoints:

```
GET /api/v1/milestones
GET /api/v1/milestones/active
POST /api/v1/milestones/activate/:id
GET /api/v1/milestones/:id/tasks
```

Database Schema:

```
CREATE TABLE milestones (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  milestone_code VARCHAR(10) NOT NULL,
  name VARCHAR(200) NOT NULL,
  phase VARCHAR(100),
  trigger_type VARCHAR(20) CHECK (trigger_type IN
('time_relative', 'event')),
  trigger_hours INTEGER,
  color VARCHAR(20),
  created_at TIMESTAMP DEFAULT NOW()
);
```

```
CREATE TABLE games (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  game_code VARCHAR(50) NOT NULL,
  home_team VARCHAR(100),
  away_team VARCHAR(100),
  venue_id UUID REFERENCES venues(id),
  kickoff_time TIMESTAMP NOT NULL,
  status VARCHAR(20),
  created_at TIMESTAMP DEFAULT NOW()
);
```

5. Evidence Service (Go + TimescaleDB)

Responsibilities:

- Task completion evidence capture
- Audit trail logging
- Photo/document storage (S3)

Endpoints:

```
POST /api/v1/evidence
GET /api/v1/evidence/task/:taskId
POST /api/v1/evidence/upload
```

TimescaleDB Schema:

```
CREATE TABLE evidence_entries (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  task_id UUID NOT NULL,
  user_id UUID NOT NULL,
  event_type VARCHAR(50), -- 'task_complete', 'task_assign',
  'status_change'
  timestamp TIMESTAMPTZ NOT NULL,
```

```

        data JSONB,
        s3_url TEXT
    );

SELECT create_hypertable('evidence_entries', 'timestamp');
CREATE INDEX idx_evidence_task ON evidence_entries(task_id,
timestamp DESC);

```

6. Notification Service (Node.js + Socket.io)

Responsibilities:

- Real-time WebSocket updates
- SMS/Email notifications
- Push notifications (mobile)

WebSocket Events:

```

// Client → Server
socket.emit('subscribe', { gameId, userId });

// Server → Client
socket.on('task_assigned', (payload) => {
    // { taskId, taskCode, description, dueTime }
});

socket.on('task_completed', (payload) => {
    // { taskId, completedBy, timestamp }
});

socket.on('dependency_resolved', (payload) => {
    // { taskId, unblockedTasks[] }
});

socket.on('milestone_activated', (payload) => {
    // { milestoneId, name, activationTime }
});

```

```
});
```

Auto-Assignment Algorithm Implementation

```
# services/auto_assigner.py
from typing import List, Optional
from models import Task, User, Role, System

class AutoAssigner:
    def __init__(self, db_session):
        self.db = db_session

    def assign_task(self, task: Task) -> Optional[User]:
        """
        Auto-assign task based on:
        1. Hat Color (system ownership)
        2. Role Permissions
        3. System Ownership
        4. Load Balancing
        """
        # Step 1: Get system hat color
        system = self.db.query(System).filter(System.id ==
task.system_id).first()
        hat_color = system.hat_color

        # Step 2: Filter users by hat color
        eligible_users = self.db.query(User).join(Role).filter(
            Role.hat_color == hat_color
        ).all()

        # Step 3: Filter by permissions
        eligible_users = [
            u for u in eligible_users
            if 'execute_task' in u.role.permissions
        ]
```



```

# Step 4: Filter by system ownership
eligible_users = [
    u for u in eligible_users
    if u.role.name in system.owner_roles
]

# Step 5: Load balance - assign to user with fewest active
tasks

if not eligible_users:
    return None # Escalate to NFL Administrator

user_task_counts = {
    user: self.db.query(Task).filter(
        Task.assigned_user_id == user.id,
        Task.status.in_(['Open', 'In Progress'])
    ).count()
    for user in eligible_users
}

assigned_user = min(user_task_counts,
key=user_task_counts.get)

# Update task
task.assigned_user_id = assigned_user.id
task.status = 'Open'
self.db.commit()

# Log to evidence
self._log_assignment(task, assigned_user)

# Notify user
self._send_notification(assigned_user, task)

return assigned_user

def _log_assignment(self, task: Task, user: User):
    """Log assignment event to evidence ledger"""
    from models import EvidenceEntry
    entry = EvidenceEntry(

```

```

        task_id=task.id,
        user_id=user.id,
        event_type='task_assigned',
        timestamp=datetime.now(),
        data={
            'task_code': task.task_code,
            'assigned_role': user.role.name,
            'auto_assigned': True
        }
    )
    self.db.add(entry)
    self.db.commit()

def _send_notification(self, user: User, task: Task):
    """Send WebSocket + SMS notification"""
    from services.notification import NotificationService
    notification_svc = NotificationService()
    notification_svc.send_task_assignment(
        user_id=user.id,
        task_id=task.id,
        task_code=task.task_code,
        description=task.description
    )

```

Dependency Resolution Engine

```

# services/dependency_resolver.py
from typing import List, Set
from models import Task, TaskDependency

class DependencyResolver:
    def __init__(self, db_session):
        self.db = db_session

    def resolve_dependencies(self, completed_task_id: str) ->
List[Task]:

```

```

    """
    When a task completes, check if any blocked tasks can be
    unblocked.
    Returns list of newly unblocked tasks.
    """
    # Find all tasks that depend on this completed task
    dependent_task_ids =
self.db.query(TaskDependency.task_id).filter(
    TaskDependency.depends_on_task_id == completed_task_id
).all()

    unblocked_tasks = []

    for (task_id,) in dependent_task_ids:
        task = self.db.query(Task).filter(Task.id ==
task_id).first()

        if task.status == 'Blocked':
            # Check if ALL dependencies are now complete
            if self._all_dependencies_complete(task):
                task.status = 'Open'
                unblocked_tasks.append(task)

            # Trigger auto-assignment
            from services.auto_assigner import AutoAssigner
            assigner = AutoAssigner(self.db)
            assigner.assign_task(task)

    self.db.commit()

    # Cascade: Check if newly unblocked tasks unblock others
    for task in unblocked_tasks:
        cascade_unblocked = self.resolve_dependencies(task.id)
        unblocked_tasks.extend(cascade_unblocked)

    return unblocked_tasks

def _all_dependencies_complete(self, task: Task) -> bool:
    """Check if all dependencies of a task are complete"""

```

```

dependencies = self.db.query(Task).join(
    TaskDependency,
    Task.id == TaskDependency.depends_on_task_id
).filter(
    TaskDependency.task_id == task.id
).all()

return all(dep.status == 'Complete' for dep in dependencies)

def get_blocking_tasks(self, task_id: str) -> List[Task]:
    """Get all tasks blocking this task"""
    blocking_tasks = self.db.query(Task).join(
        TaskDependency,
        Task.id == TaskDependency.depends_on_task_id
    ).filter(
        TaskDependency.task_id == task_id,
        Task.status != 'Complete'
    ).all()

    return blocking_tasks

def get_critical_path(self) -> List[Task]:
    """
    Find critical path - longest chain of dependencies
    This determines minimum time to completion
    """
    # Use Neo4j for efficient graph traversal
    from neo4j import GraphDatabase

    with GraphDatabase.driver(NEO4J_URI, auth=NEO4J_AUTH) as
driver:
        with driver.session() as session:
            result = session.run("""
                MATCH path = (t:Task)-[:DEPENDS_ON*]-
>(root:Task)

                WHERE NOT (root)-[:DEPENDS_ON]->()
                AND t.status <> 'Complete'
                RETURN path ORDER BY length(path) DESC LIMIT 1
            """)

```

```

        record = result.single()
        if record:
            path = record['path']
            task_ids = [node['id'] for node in path.nodes]
            return
    self.db.query(Task).filter(Task.id.in_(task_ids)).all()

    return []

```

API Integration Examples

Frontend: Task Completion with Dependency Resolution

```

// src/services/api.js
import axios from 'axios';

export const completeTask = async (taskId) => {
    try {
        // 1. Mark task complete
        const response = await axios.patch(
            `/api/v1/tasks/${taskId}/status`,
            { status: 'Complete' },
            { headers: { Authorization: `Bearer ${getToken()}` } }
        );

        // 2. Trigger dependency resolution
        const resolutionResponse = await axios.post(
            `/api/v1/dependencies/resolve`,
            { completed_task_id: taskId }
        );

        // 3. Get newly unblocked tasks
        const { unblocked_tasks } = resolutionResponse.data;
    }

```

```

    return {
      completed: response.data,
      unblocked: unblocked_tasks
    };
  } catch (error) {
    console.error('Task completion failed:', error);
    throw error;
  }
};

// src/hooks/useTasks.js
import { useState, useEffect } from 'react';
import { useWebSocket } from '../useWebSocket';
import * as api from '../services/api';

export const useTasks = (userId) => {
  const [tasks, setTasks] = useState([]);
  const socket = useWebSocket();

  useEffect(() => {
    // Initial load
    api.getTasksByUser(userId).then(setTasks);

    // Subscribe to real-time updates
    socket.on('task_assigned', (payload) => {
      if (payload.userId === userId) {
        setTasks(prev => [...prev, payload.task]);
      }
    });

    socket.on('dependency_resolved', (payload) => {
      setTasks(prev => prev.map(task => {
        if (payload.unblockedTasks.includes(task.id)) {
          return { ...task, status: 'Open' };
        }
        return task;
      }));
    });
  });
};

```

```

    return () => {
      socket.off('task_assigned');
      socket.off('dependency_resolved');
    };
  }, [userId, socket]);

const completeTask = async (taskId) => {
  const result = await api.completeTask(taskId);

  // Optimistic update
  setTasks(prev => prev.map(task => {
    if (task.id === taskId) {
      return { ...task, status: 'Complete', completedAt: new
Date() };
    }
    if (result.unblocked.some(t => t.id === task.id)) {
      return { ...task, status: 'Open' };
    }
    return task;
  }));
};

return { tasks, completeTask };
};

```

Deployment Architecture

Docker Compose (Development)

```
version: '3.8'
```

```
services:
  postgres:
    image: postgres:15
    environment:
      POSTGRES_DB: evergame
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secret
    ports:
      - "5432:5432"

  redis:
    image: redis:7
    ports:
      - "6379:6379"

  neo4j:
    image: neo4j:5
    environment:
      NEO4J_AUTH: neo4j/password
    ports:
      - "7474:7474"
      - "7687:7687"

  timescaledb:
    image: timescale/timescaledb:latest-pg15
    environment:
      POSTGRES_DB: evidence
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secret
    ports:
      - "5433:5432"

  task-service:
    build: ./services/task-service
    environment:
      DATABASE_URL: postgresql://admin:secret@postgres:5432/evergame
    ports:
      - "8001:8000"
```



```

dependency-service:
  build: ./services/dependency-service
  environment:
    NEO4J_URI: bolt://neo4j:7687
    NEO4J_AUTH: neo4j/password
  ports:
    - "8002:8000"

notification-service:
  build: ./services/notification-service
  environment:
    REDIS_URL: redis://redis:6379
  ports:
    - "8003:3000"

frontend:
  build: ./frontend
  ports:
    - "3000:3000"
  environment:
    REACT_APP_API_URL: http://localhost:8000
    REACT_APP_WS_URL: ws://localhost:8003

```

Kubernetes (Production)

```

# gda-dashboard-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: task-service
spec:
  replicas: 3
  selector:
    matchLabels:
      app: task-service
  template:

```

```

metadata:
  labels:
    app: task-service
spec:
  containers:
  - name: task-service
    image: evergame/task-service:1.0.0
    ports:
    - containerPort: 8000
    env:
    - name: DATABASE_URL
      valueFrom:
        secretKeyRef:
          name: db-credentials
          key: url
    resources:
      requests:
        memory: "256Mi"
        cpu: "250m"
      limits:
        memory: "512Mi"
        cpu: "500m"
---
apiVersion: v1
kind: Service
metadata:
  name: task-service
spec:
  selector:
    app: task-service
  ports:
  - protocol: TCP
    port: 80
    targetPort: 8000
  type: LoadBalancer

```

Performance Considerations

Caching Strategy

```
# Use Redis for frequently accessed data
import redis
from functools import wraps

redis_client = redis.Redis(host='redis', port=6379)

def cache_task_list(timeout=60):
    def decorator(func):
        @wraps(func)
        def wrapper(user_id):
            cache_key = f"tasks:user:{user_id}"
            cached = redis_client.get(cache_key)

            if cached:
                return json.loads(cached)

            result = func(user_id)
            redis_client.setex(cache_key, timeout,
                               json.dumps(result))
            return result
        return wrapper
    return decorator

@cache_task_list(timeout=30)
def get_user_tasks(user_id):
    return db.query(Task).filter(Task.assigned_user_id ==
                                  user_id).all()
```

Database Indexing

```
-- Optimize task queries
CREATE INDEX CONCURRENTLY idx_tasks_composite
ON tasks(assigned_user_id, status, milestone_id)
WHERE status IN ('Open', 'In Progress');

-- Optimize dependency lookups
CREATE INDEX CONCURRENTLY idx_task_deps_reverse
ON task_dependencies(depends_on_task_id, task_id);
```

Security Checklist

- [] Okta SSO with MFA enabled
- [] JWT tokens with short expiry (15 min) + refresh tokens
- [] Rate limiting on all API endpoints
- [] SQL injection prevention (parameterized queries)
- [] XSS protection (Content Security Policy headers)
- [] CORS configured for production domains only
- [] Encrypted database connections (SSL)
- [] Audit logging for all task completions
- [] Role-based access control enforced at API layer
- [] S3 bucket encryption for evidence uploads

Monitoring & Observability

Prometheus Metrics

```
from prometheus_client import Counter, Histogram

task_completion_counter = Counter(
    'task_completions_total',
    'Total number of task completions',
    ['system', 'milestone']
)

dependency_resolution_duration = Histogram(
    'dependency_resolution_seconds',
    'Time spent resolving dependencies'
)

@dependency_resolution_duration.time()
def resolve_dependencies(task_id):
    # ... resolution logic
    pass
```

Grafana Dashboard

- Task completion rate over time
- Dependency resolution latency
- System readiness percentages
- User assignment load distribution
- WebSocket connection count

- API response times (p50, p95, p99)

Next Steps: Implement Phase 1 (Foundation) from the GDA Orchestration Playbook roadmap.